# Multimodal Multiobjective Neural Architecture Search for Lightweight and Failure-Resilient Time Series Forecasting

Yifan Li, Hong Zhao, *Member, IEEE*, Jian-Yu Li, *Member, IEEE*, Jing Liu, *Senior Member, IEEE*

***Abstract*—Time series forecasting (TSF) plays a pivotal role in decision-making and risk mitigation by predicting future values from historical observations. Despite significant improvements in forecasting accuracy, existing TSF models face two critical challenges: increasing computational complexity hinders deployment on resource-constrained platforms, and vulnerability to random failures compromises prediction stability. To address these issues, we propose M3NAS-TSF, a multimodal multiobjective neural architecture search framework that automates the design of lightweight and failure-resilient TSF models. Specifically, to address computational complexity, we introduce a modular time series super-net that defines a flexible and compact search space, enabling the discovery of architectures with reduced model size. To enhance failure resilience, we develop a multimodal multiobjective neural architecture search algorithm that promotes architectural diversity through an architecture-aware niching strategy and an architectural diversity distance metric, ensuring a broad set of robust candidate models. We also propose a node distribution heatmap and a structural entropy index to assess architectural diversity without ground-truth Pareto sets. Extensive experiments on 40 forecasting cases demonstrate that M3NAS-TSF outperforms six representative baselines, achieving superior forecasting accuracy (the maximum reduction in the RMSE reached 14.50%) with a smaller model size while maintaining greater architectural diversity for failure resilience.**

***Index Terms*—Time series forecasting, neural architecture search, multimodal multiobjective optimization.**

## I. Introduction

Time series forecasting (TSF) is critical across many domains, such as economics and finance [1], disease propagation analysis [2], and traffic flow [3], where decision-makers are often interested in forecasting future values or potential hazardous events based on historical observations.

Over the past few decades, many efforts have been made to improve the forecasting accuracy of TSF [4]. Existing TSF models become increasingly cumbersome as hardware storage capabilities improve and the amount of available data continues to increase. For instance, the recent LSTNet [5] and Informer [6] utilize 74.42G and 41.07G floating-point operations per second (Flops) as well as 0.07M and 11.29M parameters, respectively, to achieve high forecasting performance. These requirements are much larger than those of the early LSTM with a linear layer (approximately 1.08G Flops and 1344 parameters). Such increasing model sizes and computational costs result in heavy inference overhead, which poses serious challenges for deploying TSF models on embedded or resource-constrained systems. Model compression and compact model design are two straightforward ideas for addressing complexity and efficiency issues. However, although existing off-the-shelf compression techniques, including pruning and quantization, can reduce model complexity, they inevitably degrade the forecasting performance because of information loss. In addition, handcrafting new compact and efficient models is expensive from an engineering standpoint and relies heavily on human expertise and experience. As such, researchers struggle to analyse how the parameter settings affect the performance of available models.

Furthermore, during the time series forecasting process, various uncontrollable or irregular random failures may be encountered, which can significantly affect the stability and accuracy of forecasting models. These failures may arise from sensor malfunctions, malicious attacks, or soft errors caused by cosmic radiation and radioactive impurities. In safety-critical and resource-constrained domains, random bit-flips due to high-energy particles have been widely reported to cause silent data corruption, control flow divergence, or even system crashes [7] [8]. For example, a single soft error in a flight control system can lead to major mission failure, and in large-scale cloud data centres, soft errors have been shown to corrupt computation results without immediate detection. To address such failures, maintaining a diverse pool of backup models is an effective fault-tolerant strategy. When a forecasting model underperforms because of errors, noise, or targeted attacks, switching to an alternative model with a different architecture or comprehensively utilizing diverse models can ensure the continuity and robustness of prediction. This point highlights the importance of architectural diversity in TSF, especially in real-world deployments subject to unpredictable disturbances.

For the above reasons, this paper proposes a multimodal multiobjective neural architecture search (NAS) framework to develop forecasting models that meet the above conditions, called M3NAS-TSF. First, to address the problem of architectural incompatibility between time series data and the

Received 26 August 2025; revised 1 December 2025; accepted 21 January 2026. This work was supported in part by the Key Project of Science and Technology Innovation 2030 supported by the Ministry of Science and Technology of China under Grant 2018AAA0101302 and in part by the General Program of National Natural Science Foundation of China (NSFC) under Grant 61773300. (*Corresponding authors: Hong Zhao*)

Yifan Li is with the School of Artificial Intelligence, Capital University of Economics and Business, Beijing 100070, China (email: 18066899461@163.com).

Hong Zhao and Jing Liu are with the Guangzhou Institute of Technology, Xidian University, Guangzhou 510555, China (email: hongzhao@xidian.edu.cn; neouma@163.com).

Jian-Yu Li is with the College of Artificial Intelligence, Nankai University, Tianjin 300350, China (email: jianyuli@nankai.edu.cn).

standard super-net in one-shot NAS, we propose a modular time series super-net (MoTS-Net), which defines a flexible and compact architecture space tailored to efficient adaptive feature extraction. Second, a multimodal multiobjective neural architecture search algorithm (M3NAS) is proposed to search for lightweight and high prediction accuracy Pareto networks on the trained MoTS-Net. Moreover, the two key operators of M3NAS are an architecture-aware niching selection (AANS) strategy and an architectural diversity distance (ADD) metric, which effectively improve the diversity of architectures searched by vanilla nondominated sorting genetic algorithm-II (NSGA-II) [9] without degrading the performance of the Pareto networks. Finally, a node distribution heatmap (NDH) and a structural entropy (SE) index are proposed to analyse the diversity of architectures without reference to the true Pareto sets.

In contrast to prior NAS methods, our proposed M3NAS-TSF introduces several key distinctions. First, the proposed MoTS-Net is explicitly tailored for the characteristics of time series data and moves beyond the large scale image data-centric super-net structures of prior works [10] [11]. Second, existing one-shot NAS methods [12] [13] typically rely on standard multiobjective optimization algorithms such as NSGA-II [9], which are limited to promoting architectural diversity. In contrast, our proposed M3NAS is a dedicated multimodal multiobjective optimizer that explicitly fosters architectural diversity through the AANS strategy and the ADD metric, which contributes to enhancing failure resilience in real-world deployments. Consequently, M3NAS-TSF does not merely search for accurate and compact models but automatically discovers a set of high-performing, lightweight architectures that are robust to random failures, thereby addressing a critical gap in the deployment of reliable TSF models on resource-constrained platforms. In summary, the contributions of this paper can be summarized as follows:

1) A MoTS-Net is proposed and successfully applied to TSF. It pioneers the exploration of one-shot NAS on time series-related tasks, facilitating the development of automated TSF models.
2) An M3NAS algorithm including an AANS strategy and an ADD metric is designed to search Pareto networks on the trained MoTS-Net, which provides decision-makers with more diverse network architectures without degrading the performance of the Pareto networks.
3) An NDH and an SE index are proposed for effectively estimating the characteristics of network architectures. For nonbenchmark optimization problems such as NAS, the true Pareto set is difficult to obtain, while the NDH and SE can conveniently measure the diversity of the architectures considering the inherent properties of MoTS-Net.

The remainder of this paper is organized as follows. Section II introduces the related background of this work. Section III presents the details of the proposed M3NAS-TSF. The experimental design and the results are presented in Sections IV and V, respectively. Section VI presents the conclusion of this work.

## II. Background

### A. Time Series Forecasting

The purpose of TSF is to use the observed time series in the past to forecast the unknown series in a look-ahead horizon [14]. Under the rolling forecasting setting with a fixed window size, we have the input $X = \{x_i^j \in \mathbb{R} | i = 1, 2, \cdots, L_x, j = 1, 2, ..., N_f\}$, and the output is to forecast the corresponding series $Y = \{y_i \in \mathbb{R} | i = 1, 2, \cdots, L_y\}$, where $L_x$ and $L_y$ represent the lengths of the input and the output, respectively. $N_f$ is the dimension (i.e., the number of features) of the input, and $\mathbb{R}$ is the real number space. When $N_f = 1$ and $N_f > 1$, the TSF task is in the univariate forecasting mode and the multivariate forecasting mode, respectively.

The mainstream algorithms of TSF can be divided into statistical-based methods and deep learning (DL)-based methods. Statistical-based methods, such as the autoregressive integrated moving average model, support vector regression, and the gradient boosting regression method, have appealing properties, such as interpretability and theoretical guarantees. Nevertheless, these models require strict statistical properties such as the stationarity of the data, making it difficult to achieve high-accuracy forecasts on complex data. At present, the emergence of DL techniques [15] [16] greatly alleviates this dilemma. For instance, Lai et al. [5] used a convolutional neural network (CNN) and a recurrent neural network (RNN) to extract the short-term local dependency among features and discover the long-term dependency of time series trends, respectively. A multichannel residual block in parallel with an asymmetric structure based on a deep CNN [17] was subsequently proposed to determine the multivariate time series dependence of the aperiodic data. Recently, Zhou et al. [18] proposed the DeepHydro model, which captures temporal dependence in coevolving time series with a conditioned latent RNN.

Although recent DL-based methods have significantly improved forecasting accuracy, the TSF task is essentially a regression problem and does not always benefit from large models. In many cases, deeper networks introduce redundancy and unnecessary computational overhead, making them unsuitable for deployment on resource-constrained platforms. To address this issue, we propose to automatically design lightweight TSF models via neural architecture search, thus enabling efficient execution on devices with limited computational and memory resources.

### B. Multimodal Multiobjective Optimization

Many real-world engineering applications consider optimizing multiple objectives simultaneously, and there is a conflict between objectives, which means that no solution can obtain the best performance on all the objectives. Such problems are called multiobjective optimization problems. Furthermore, another kind of multiobjective optimization problem arises in which multiple solutions in the decision space share the same or similar objective values. Such problems are named multimodal multiobjective optimization problems (MMOPs) [19]. Solving MMOPs can not only reveal the essential properties of problems to better understand them but also directly provide

users with multiple solutions in different scenarios to address unpredictable random failures.

Research in multimodal multiobjective optimization is undergoing a significant paradigm shift, evolving from fundamental niching frameworks towards an emphasis on data-driven methodologies [20]–[22]. The seminal work of [23] established a stable niching structure using ring topology and dual-space crowding distance, laying the groundwork for subsequent algorithms. Building on this foundation, [24] extended the scope of solution sets by explicitly searching for local Pareto optima. As research has progressed, [25] introduced a scaled niche distance to coordinate diversity in objective and decision spaces, while [26] employed hierarchical clustering and ranking to mitigate convergence degradation caused by excessive diversity preservation. Concurrently, data-driven approaches have gained prominence. For example, [27] utilized graph neural networks to learn the Pareto set topology for offspring generation, and [28] used an integrated self-supervised support vector machine to simultaneously account for convergence and diversity in the selection process.

While existing multimodal multiobjective optimization algorithms have been shown to be effective at preserving diversity in both decision and objective spaces, they are generally designed for standard optimization problems with simple decision variable structures. In contrast, NAS involves a highly discrete search space, where each solution represents architectural components with complex dependencies. Moreover, unlike conventional MMOPs with explicit and low-cost objective evaluations, NAS typically requires partial or full network training to assess forecasting accuracy and model efficiency, leading to expensive fitness evaluations. To address these challenges, we design a MoTS-Net to construct a modular and lightweight search space tailored for TSF, and adopt a one-shot NAS method to reduce the evaluation overhead via weight sharing. Additionally, AANS and ADD are proposed to maintain the architectural diversity that is essential to MMOPs.

### C. Neural Architecture Search

Neural architecture search aims to find a network architecture that achieves optimal performance using limited computing resources in an automated manner with minimal human intervention. Early NAS looks for the optimal neural network in a nested manner [29], i.e., many networks are sampled and trained from scratch, whose computational cost is unaffordable on large datasets. Recent approaches adopt a weight sharing mechanism to reduce the number of computations, i.e., first constructing a super-net subsuming many subnetworks, then searching for the optimal subnetwork on the trained super-net [30]. This super-net is trained only once, and each subnetwork directly inherits corresponding weights from the super-net. One-shot NAS [31] is a typical representation that uses the weight sharing mechanism and has attracted the attention of many researchers. For example, Guo et al. [32] proposed single path one-shot NAS with uniform sampling that further eased the training process. Later, Chu et al. [13] developed a unified approach for multipath one-shot NAS with the help of shadow batch normalization. In addition, some studies have been devoted to improving the efficiency of super-net training [12].

However, the above one-shot NAS methods focus on the construction of a super-net, and use only a vanilla evolutionary algorithm (EA) in the subnetwork search stage. For example, GreedyNAS [12] uses NSGA-II [9] to embed hardware constraints into the evolutionary process, and MixPath [13] adopted NSGA-II to maximize the image classification accuracy and minimize the number of Flops. Since the elite selection in NSGA-II selects the next generation of the population only according to the performance in the objective space, it focuses only on the diversity and convergence of the searched subnetworks in the objective space, ignoring the diversity of the subnetworks in the decision space. Moreover, in real-world scenarios, various uncontrollable random failures may be encountered, which will affect the stability and accuracy of the models. If more diverse networks in terms of the decision space can be provided, when random failures occur, we can quickly switch to other networks or integrate multiple networks to ensure the continuity of the current task while maintaining good performance [33]. As a result, it is necessary to design a multimodal multiobjective optimization algorithm in the search stage of one-shot NAS to improve the diversity of the searched subnetworks in the decision space.

To further clarify the differences between our proposed M3NAS-TSF and existing NAS methods, we summarize the key characteristics of several representative algorithms in Table I. Most prior methods are tailored for the computer vision (CV) task, focus on accuracy-only objectives, and neglect architectural diversity and robustness. In contrast, our M3NAS-TSF introduces a task-specific super-net tailored for time series forecasting. It also incorporates a multimodal multiobjective optimization algorithm that explicitly considers architectural diversity and model size, aiming to improve failure tolerance and enhance deployment capability on resource-limited platforms.

## III. M3NAS-TSF

### A. Framework of M3NAS-TSF

After the review of Section II, we tried to use one-shot NAS to search for lightweight and high-accuracy TSF models while ensuring high architectural diversity. However, this idea poses two key challenges are as follows. 1) Architectural incompatibility between existing super-net designs and time series data. While one-shot NAS has been widely adopted in CV, its super-net structures are typically designed for high-dimensional image inputs (e.g., $3\times320\times320$), leveraging spatial hierarchies and two-dimensional locality. In contrast, time series inputs are temporally ordered, lower-dimensional (e.g., $1\times50$), and contain long-range dependencies that standard image-oriented convolutional operations struggle to capture. Directly applying existing CV-based super-nets to TSF can cause input mismatches, ineffective feature extraction, and suboptimal representations. 2) Lack of architectural diversity in the evolutionary search stage. Most existing one-shot NAS frameworks adopt a vanilla evolutionary algorithm to search for candidate architectures. These EAs typically focus on

TABLE I
COMPARISON OF M3NAS-TSF AND REPRESENTATIVE NAS METHODS.

| NAS methods | Applications Fields | Search strategies | Optimization objectives | | Diversity consideration | | Robustness to failures |
|---|---|---|---|---|---|---|---|
| | | | Accuracy | Model size | Objective space | Decision space | |
| ENAS [10] | CV | Reinforcement learning | ✓ | ✗ | ✗ | ✗ | ✗ |
| DARTS [11] | CV | Gradient-based | ✓ | ✗ | ✗ | ✗ | ✗ |
| GreedyNAS [12] | CV | NSGA-II | ✓ | ✓ | ✓ | ✗ | ✗ |
| MixPath [13] | CV | NSGA-II | ✓ | ✓ | ✓ | ✗ | ✗ |
| SPOS [32] | CV | Genetic algorithm | ✓ | ✗ | ✗ | ✗ | ✗ |
| dMA-NAS-UTSF [34] | TSF | Memetic algorithm | ✓ | ✗ | ✗ | ✗ | ✗ |
| **M3NAS-TSF (Ours)** | TSF | Multimodal multiobjective EA | ✓ | ✓ | ✓ | ✓ | ✓(Architectural diversity for failure tolerance) |

convergence and diversity in the objective space (e.g., accuracy and model size) while ignoring architectural diversity in the decision space. As a result, the final Pareto front may contain architectures that are vulnerable to the random failure, reducing the robustness and adaptability of the resulting models.

To overcome these problems, we first use multiple encoding techniques to transform the time series into images as the input of the designed MoTS-Net and then propose the M3NAS algorithm to search diverse networks on the trained MoTS-Net. Specifically, the M3NAS-TSF process is shown in Algorithm 1. The input $X = \{x_i^j | i = 1, 2, \ldots, L_x, j = 1, 2, \ldots, N_f\}$ is first divided into a training/validation/test set $(X_{train}, X_{valid}, X_{test})$ at a ratio of 6:2:2 (line 1). Then, if $N_f > 1$, $X_{train}, X_{valid}, X_{test}$ are separately flattened along the direction of the features (lines 2-3) and in turn transformed into the 2-D images $(I_{train}, I_{valid}, I_{test})$ through four time series encoding strategies (line 4). If $N_f = 1$, $X_{train}, X_{valid}, X_{test}$ are directly transformed into 2-D images (line 4). Next, MoTS-Net is constructed and trained by $I_{train}$ to obtain the weights with the smallest loss on $I_{valid}$ (line 5), where $\mathcal{L}$ and $W_{\mathcal{L}}$ represent the architecture and the weights of MoTS-Net, respectively. The trained MoTS-Net can subsequently be regarded as an evaluator of the subnetworks. Each subnetwork in MoTS-Net can be considered as an individual of M3NAS, and the optimization objectives are to minimize the forecasting error and the number of parameters of the subnetworks. Here, the M3NAS algorithm is proposed to search for Pareto networks on the trained MoTS-Net, whose key operations include population initialization (line 6), AANS (line 11), crossover (line 12), mutation (line 15), fast nondominated sorting (line 18), ADD calculation (line 19), and elite selection (line 20).

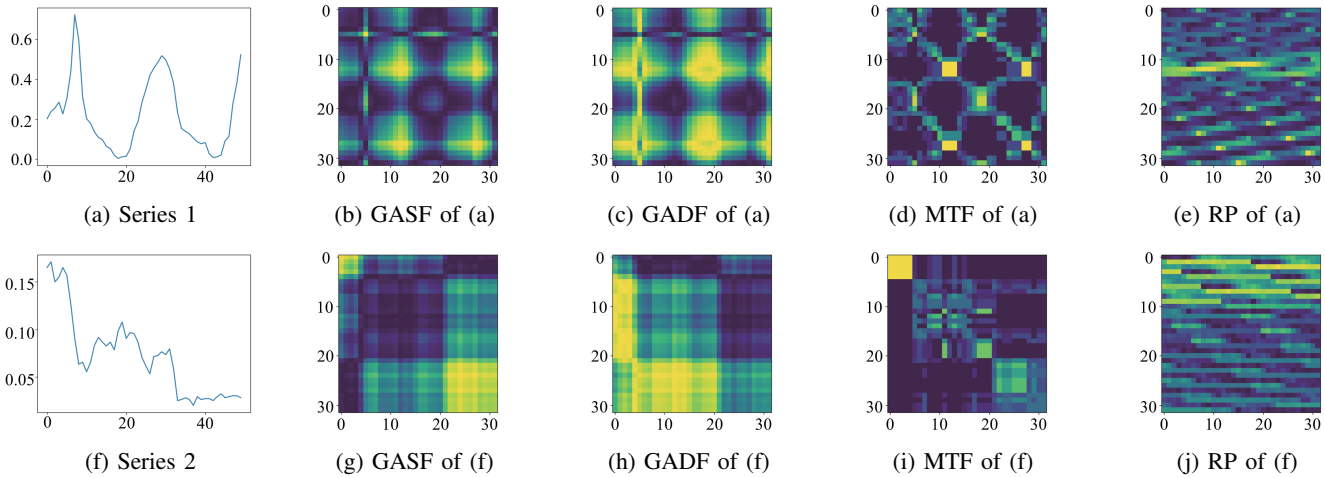
**Algorithm 1** Framework of M3NAS-TSF.

**Input:** The original time series $X = \{x_i^j | i = 1, 2, \ldots, L_x, j = 1, 2, \ldots, N_f\}$, input image size $S_{im}$, crossover probability $p_c$, mutation probability $p_m$, population size $N$, maximum evolutionary generations $g$, crowding factor $F_c$.
**Output:** The final networks.

1: $X_{train}, X_{valid}, X_{test} \leftarrow$ divide $X$ in a ratio of 6:2:2;
2: **If** $N_f > 1$:
3: $X_{train}, X_{valid}, X_{test}$ are separately flatten along the direction of the features;
4: $I_{train}, I_{valid}, I_{test} \leftarrow$ piecemeal transform $X_{train}, X_{valid}, X_{test}$ into 4-channel $S_{im} \times S_{im}$ images by time series encoding strategies;
5: $\mathcal{L}, W_{\mathcal{L}} \leftarrow$ construct and train the MoTS-Net by $I_{train}, I_{valid}$;
6: $pop \leftarrow$ Initialize population $\{p^k | k = 1, 2, \cdots, N\}$ by Latin hypercube sampling on $\mathcal{L}$, and evaluate $p^k$ by the inherited weights from $W_{\mathcal{L}}$;
7: $g_c \leftarrow 0$;
8: **While** $g_c < g$:
9: $pop_c \leftarrow \varnothing, pop_m \leftarrow \varnothing$;
10: **for** $i$ in range$([1, int(N/2)])$:
11: $p^1, p^2 \leftarrow$ AANS $(pop, F_c)$;
12: $p^{c1}, p^{c2} \leftarrow$ single-point crossover $(p^1, p^2)$ and evaluate $p^{c1}, p^{c2}$;
13: $pop_c \leftarrow pop_c \cup p^{c1} \cup p^{c2}$;
14: **for** $p$ in $pop_c$:
15: $p^m \leftarrow$ single-point mutation $(p)$ and evaluate $p^m$;
16: $pop_m \leftarrow pop_m \cup p^m$;
17: $pop_{combined} \leftarrow pop \cup pop_c \cup pop_m$;
18: $pop_{combined} \leftarrow$ fast nondominanted sort $(pop_{combined})$;
19: $pop_{combined} \leftarrow$ ADD $(pop_{combined})$;
20: $pop \leftarrow$ next generation population selection $(pop_{combined})$;
21: **Return** $pop$.

### B. Encoding Time Series into Images

Inspired by the recent success of deep learning in computer vision [35] [36], many researchers encode the 1-D time series as 2-D images [37] [38] to facilitate the use of operators in CV. The prevailing encoding methods include the Gramian angular summation/difference field (GASF/GADF) [39], the Markov transition field (MTF) [39], and the recurrent plot (RP) [40], whose examples are shown in Fig. 1. Specifically, GASF and GADF capture global temporal correlations by encoding the angular relationships among different time steps, preserving temporal dependency through the quasi-Gramian matrix structure. MTF encodes the multispan first-order transition probabilities between quantized temporal states by spreading the Markov transition matrix across time, thereby capturing both temporal patterns and potential periodic structures. RP reveals repetitive patterns and state transitions in the phase space, making it effective for uncovering underlying dynamics



(a) Series 1 (b) GASF of (a) (c) GADF of (a) (d) MTF of (a) (e) RP of (a)

(f) Series 2 (g) GASF of (f) (h) GADF of (f) (i) MTF of (f) (j) RP of (f)

Fig. 1. Two examples of the original time series and the corresponding encoded images.

and irregular recurrences. Compared with other encoding approaches, such as short-time Fourier transform and continuous wavelet transform, these methods yield fixed-size, low-noise, and topology-preserving images that are better aligned with convolutional processing. Moreover, their deterministic and interpretable characteristics facilitate complementary representations of temporal dynamics and downstream architecture search. As a result, our work encodes the time series as images by the above four techniques, and each type of image is regarded as one channel of the MoTS-Net input.

### C. MoTS-Net

In this subsection, the MoTS-Net paradigm is first introduced, which defines a modular feedforward pipeline for each subnetwork. Afterwards, the specific settings in the MoTS-Net are introduced in sequence. Finally, the training approach and advantages of MoTS-Net are presented.

**Modular Design Paradigm of MoTS-Net**. MoTS-Net follows a hierarchical modular design paradigm, which decomposes the super-net into reusable, self-contained functional nodes organized into searchable layers. As shown in Fig. 2, each subnetwork is constructed by selecting exactly one node per layer, forming a single path through the super-net. This modular design ensures rich node combination flexibility, where each subnetwork is assembled by mixing and matching nodes across layers, enabling exponential candidate subnetworks (e.g., $5^8 = 390,625$ possible subnetworks for an 10-layer super-net with 5 nodes per layer).

**Layerwise Modularization Design**. Taking the total number of layers $w = 10$ as an example, the detailed configuration of each layer in MoTS-Net is listed in Table II, where $L_1$ and $L_{10}$ are fixed layers. $L_1$ employs a fixed 3×3 convolution to reliably extract primary spatial features from the input data and ensure stable input dimensions for subsequent layers, while $L_{10}$ is a fully connected layer guaranteeing consistent output shapes across all subnetworks. $L_2$ to $L_9$ are modular and searchable layers, each containing a set of candidate functional nodes. Such layerwise modularization design simplifies super-net training and subnetwork evaluation through decoupled layerwise selection, thereby enabling efficient combination search over layer-specific operations.

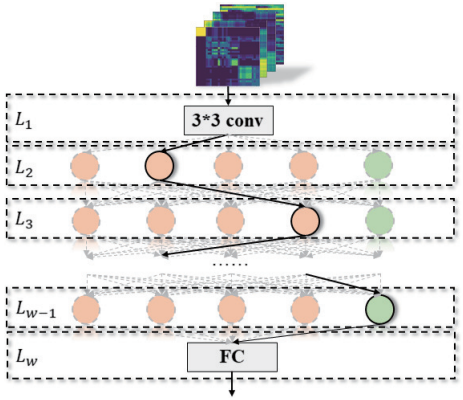


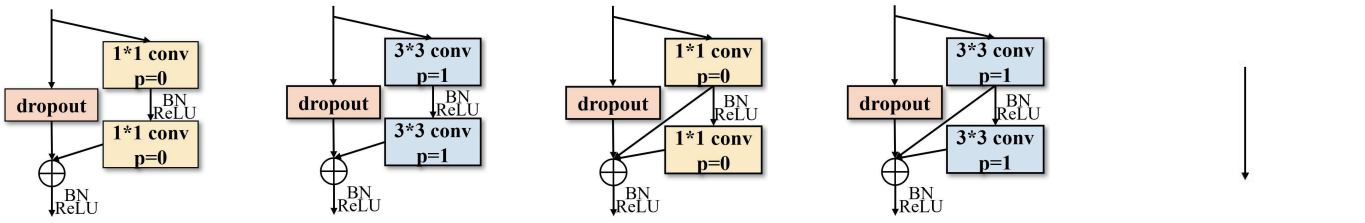

Fig. 2. The framework of MoTS-Net, which takes four types of encoded images as the input and includes 2 fixed layers ($L_1$ and $L_w$) and $w-2$ searchable layers ($L_2$ to $L_{w-1}$). The solid black line is an example of a path, and the nodes that pass through form a candidate TSF network.

**Design of Functional Nodes**. As shown in Figs. 3 and 4, two types of candidate nodes ($CN^1$ and $CN^2$ in Table II) with different strides are defined to support an expressive search space. 1) Kernel size search: Available convolutional kernels include 1×1 and 3×3. 2) Feature transformation search: Candidate transformations include shortcut connections, dense connections, identity mappings, and standard convolutions. These nodes allow the network to adaptively assemble rich temporal feature extractors with varying complexity, enabling fine-grained control over computational cost and representational capacity.

Based on the above construction of MoTS-Net, taking $w = 10$ as an example, the path of a candidate network can be exemplified as $[4, 1, 2, 1, 0, 0, 3, 2]$, where each element represents the node number of the corresponding searchable layer, i.e., $L_1$: 3*3 conv, $L_2$: node 4 of $CN^2$, $L_3$: node 1 of $CN^1$, $L_4$: node 2 of $CN^2$, $L_5$: node 1 of $CN^1$, $L_6$: node 0

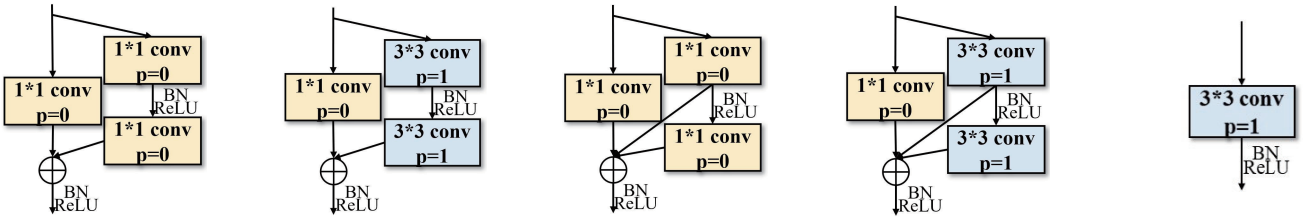
TABLE II
MODULAR DESIGN OF MOTS-NET TAKING $w = 10$ AS AN EXAMPLE. "$CN^1$" AND "$CN^2$" DENOTE THE CANDIDATE NODES WITH STRIDE 1 AND 2, RESPECTIVELY. "$FC$" IS THE FULLY CONNECTED OPERATION.

| **layer** | $L_1$ | $L_2$ | $L_3$ | $L_4$ | $L_5$ | $L_6$ | $L_7$ | $L_8$ | $L_9$ | $L_{10}$ |
|---|---|---|---|---|---|---|---|---|---|---|
| **input shape** | $32 \times 32$ | $16 \times 16$ | $16 \times 16$ | $8 \times 8$ | $8 \times 8$ | $4 \times 4$ | $4 \times 4$ | $2 \times 2$ | $2 \times 2$ | $1 \times 1$ |
| **input channels** | 4 | 8 | 8 | 16 | 16 | 32 | 32 | 64 | 64 | 128 |
| **stride** | 2 | 2 | 1 | 2 | 1 | 2 | 1 | 2 | 1 | 1 |
| **node types** | 3*3 conv | $CN^2$ | $CN^1$ | $CN^2$ | $CN^1$ | $CN^2$ | $CN^1$ | $CN^2$ | $CN^1$ | $FC$ |
| **out channels** | 8 | 16 | 16 | 32 | 32 | 64 | 64 | 128 | 128 | $L_y$ |




(a) 0: shortcut_k1_s1 (b) 1: shortcut_k3_s1 (c) 2: dense_k1_s1 (d) 3: dense_k3_s1 (e) 4: Identity mapping

Fig. 3. Five candidate nodes of $CN^1$ with stride=1.




(a) 0: shortcut_k1_s2 (b) 1: shortcut_k3_s2 (c) 2: dense_k1_s2 (d) 3: dense_k3_s2 (e) 4: 3×3 conv

Fig. 4. Five candidate nodes of $CN^2$ with stride=2.

of $CN^2$, $L_7$: node 0 of $CN^1$, $L_8$: node 3 of $CN^2$, $L_9$: node 2 of $CN^1$, $L_{10}$: the fully connected operation.

**Weight-sharing Optimization for MoTS-Net**. As formulated in (1), MoTS-Net is trained on $I_{train}$ using the mean square error loss, and the weights $W_{\mathcal{L}}$ are optimized to minimize the validation loss $loss_{valid}$ on $I_{valid}$. The training process adopts a uniform sampling strategy consistent with that of [32]. Once trained, MoTS-Net serves as a shared-weight evaluator for subnetwork performance during the following architecture search.

$$W_{\mathcal{L}} = \arg\min_{d} \{loss_{valid}(\mathcal{L}, W)\} \tag{1}$$

**Advantages of MoTS-Net**. There are two major advantages of the proposed MoTS-Net. First, time series-encoded images are taken as inputs, allowing the direct integration of residual [41] and dense [42] connections from CV to mitigate gradient vanishing and explosion issues in deep models. Second, its modular structure provides a rich combination of lightweight yet expressive components, such as identity mappings and convolutions with dense connections, resulting in the search space containing networks ranging from 0.09M to 0.6M parameters and 0.25G to 1.4G Flops. Compared with handcrafted networks, MoTS-Net offers greater structural flexibility and a lightweight search space, enabling M3NAS to successfully identify efficient and effective TSF models.

### D. M3NAS

To address the lack of architectural diversity in the evolutionary search stage of one-shot NAS, we propose M3NAS, a diversity-driven extension of the classical NSGA-II algorithm [9]. Unlike conventional multiobjective EAs that emphasize convergence and diversity in the objective space, M3NAS is explicitly designed to promote architectural diversity among subnetworks in the decision space, which is critical for fault tolerance and decision flexibility in real-world scenarios. The key innovations of M3NAS are the architecture-aware niching selection strategy and the architectural diversity distance, which replace the parent selection and crowding distance calculation in NSGA-II, respectively. Specifically, M3NAS consists of six major components: population initialization, AANS, genetic operators, fast nondominated sorting, ADD calculation, and elite selection. In this subsection, we first introduce the XOR-distance and $\tau$-distance metrics used in AANS and ADD and then present the full process of the proposed M3NAS algorithm.

**Definition 1** (XOR-distance): The XOR-distance is defined as the sum of elementwise bitwise exclusive OR operations between two network paths. For instance, given two

paths $[4,1,2,1,0,0,3,2]$ and $[2,0,1,2,3,4,3,2]$, their XOR-distance is calculated as $6+1+3+3+3+4+0+0=20$. This metric provides a fast and coarse-grained measure of architectural dissimilarity by treating the network path as an integer vector. It emphasizes the degree of change at each layer without requiring prior knowledge of node semantics to quickly encourage architectural diversity.

**Definition 2 ($\tau$-distance)**: The $\tau$-distance is defined as the sum of pairwise distances between corresponding nodes of two network paths. The pairwise distance values are drawn from a predefined reference table (see Fig. 5), which is constructed based on three key architectural factors: operation type, kernel size, and the presence of trainable parameters, where larger distances indicate greater functional dissimilarity between node operations. For example, the $\tau$-distance between $[4,1,2,1,0,0,3,2]$ and $[2,0,1,2,3,4,3,2]$ is $2+1+2+2+2+1+0+0=10$. Unlike the XOR-distance which reflects purely numeric deviation, the $\tau$-distance captures the functional dissimilarity of nodes, making it a more semantically meaningful metric.

| | 0 | 1 | 2 | 3 | 4 |
|---|---|---|---|---|---|
| 0 | 0 | 1 | 1 | 2 | 1 |
| 1 | 1 | 0 | 2 | 1 | 1 |
| 2 | 1 | 2 | 0 | 1 | 2 |
| 3 | 2 | 1 | 1 | 0 | 2 |
| 4 | 1 | 1 | 2 | 2 | 0 |

Fig. 5. The reference distance table. The shaded area represents the distance of each pair of nodes in $[4,1,2,1,0,0,3,2]$ and $[2,0,1,2,3,4,3,2]$.

**Population Initialization**. In M3NAS, a network path corresponds to the code of a candidate individual. The number of parameters and the forecasting error are the performance indicators of the individuals. Here, Latin hypercube sampling is performed on the candidate nodes for the searchable layers to initialize the population $\{p^k|k=1,2,\cdots,N\}$, where $p^k$ is the $k$-th individual and its number of parameters is determined by the nodes in its path, and its forecasting error is obtained by inheriting the weights $W_{\mathcal{L}}$ of MoTS-Net.

**AANS**. Traditional selection strategies such as roulette wheel selection tend to conduct intensive searches around individuals with optimal performance in the objective space, easily leading to convergent offspring architectures. During the parent selection phase of the EA, AANS partially shifts the search pressure from relying solely on objective space performance towards diversity in the decision space. Specifically, the use of the XOR-distance and $\tau$-distance mandates the consideration of individuals that are architecturally most different from a randomly chosen individual, thereby ensuring that the two parent individuals are sufficiently dispersed in the decision space. As shown in Fig. 6, we first randomly select an individual ($\mathscr{A}$) in the current population and then arbitrarily choose $\lceil F_c \cdot N \rceil$ individuals from the same population, termed CF individuals, where $F_c$ is a user-defined hyperparameter referred to as the crowding factor (set to 0.3 in Fig. 6 for example). Next, for each CF individual, we compute both the XOR-distance and the $\tau$-distance to $\mathscr{A}$. Afterwards, the CF individual exhibiting the minimal XOR-distance ($\mathscr{C}$) and the one exhibiting the minimal $\tau$-distance ($\mathscr{D}$) are identified. These two individuals, together with $\mathscr{A}$, form a tentative parent pool. Finally, the two highest-performing individuals within this pool are retained as the parents for the subsequent genetic operation, thereby ensuring diversity in offspring generation.

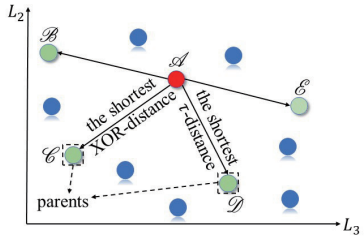



Fig. 6. Illustration of the AANS strategy taking two searchable layers as an example, where $\mathscr{A}$ is an arbitrarily chosen individual. $\mathscr{B},\mathscr{C},\mathscr{D},\mathscr{E}$ are the CF individuals of $\mathscr{A}$. $\mathscr{C}$ and $\mathscr{D}$ are the individual with the shortest XOR-distance and the individual with the shortest $\tau$-distance, respectively.

**Genetic Operators**. We employ single-point crossover and single-point mutation as the genetic operators. For example, given two parent paths $[4,1,2,1,0,0,3,2]$ and $[2,0,1,2,3,4,3,2]$, the crossover operator randomly selects a crossover point and exchanges the path segments after this point to generate offspring $[4,1,2,2,3,4,3,2]$ and $[2,0,1,1,0,0,3,2]$, while the mutation operator alters a randomly chosen node to yield $[4,1,2,2,3,1,3,2]$ and $[2,0,2,1,0,0,3,2]$. Then, the performance of the offspring is calculated by the parameters of their nodes and the inherited weights from $W_{\mathcal{L}}$.

**Fast Nondominated Sorting**. The current population is combined with the offspring to obtain the combined population, and the combined population is divided into different frontiers using fast nondominated sorting [9].

**ADD Calculation**. The crowding distance in NSGA-II is used during the environmental selection phase to select more unique individuals in the objective space. In contrast, the ADD metric using XOR-distance and $\tau$-distance measures the isolation degree of an individual in the decision space and is used to directly replace the crowding distance in NSGA-II. An individual with a high ADD value has no architecturally similar neighbours within the group of individuals with similar performance in the objective space. Prioritizing the retention of these individuals during environmental selection directly protects those unique network architectures that might be eliminated by the traditional objective-space crowding distance. The specific calculation process of ADD is shown in Algorithm 2. If the number of individuals in the $i$-th frontier is less than 3 (line 2), the ADD of these individuals is infinite (line 4). Otherwise, for each individual $p$ in the $i$-th frontier (line 6), we calculate the XOR-distance and $\tau$-distance between $p$ and the other individuals, marked as $XOR_p$ and $\tau_p$ (lines 7-8), respectively, and then normalize $XOR_p$ and $\tau_p$

(line 9). Finally, the ADD of $p$ is the mean of the minimum $XOR_p$ and the minimum $\tau_p$ (line 10).

**Algorithm 2** ADD.

**Input:** The combined population $pop_{combined} = \{pop_{f_1}, pop_{f_2}, \cdots, pop_{f_s}\}$, where $pop_i, i = f_1, f_2, \cdots, f_s$ are the individuals in $i$-th frontier.
**Output:** $pop_{combined}$ with the ADD.

1: **for** $i$ in range($f_s$):
2: **if** the number of individuals in $pop_i < 3$:
3: **for** $p$ in $pop_i$:
4: ADD of $p \leftarrow$ infinite;
5: **else**:
6: **for** $p$ in $pop_i$:
7: $XOR_p \leftarrow$ calculate the XOR-distance of $p$ and other individuals in $pop_i$;
8: $\tau_p \leftarrow$ calculate the $\tau$-distance of $p$ and other individuals in $pop_i$;
9: $XOR_p, \tau_p \leftarrow$ Min-Max Normalization $(XOR_p, \tau_p)$;
10: ADD of $p \leftarrow$ 0.5*(min($XOR_p$)+min($\tau_p$));
11: **Return** $pop_{combined}$.

**Elite Selection**. The individuals in the first $f_s - 1$ frontiers are directly selected to enter the next generation, where $f_s$ is the total number of frontiers. Particularly for the $f_s$-th frontier, the top $N - \sum_{i=1}^{f_s-1} |pop_i|$ individuals with the largest ADD in $pop_{f_s}$ are also selected to join the next generation population.

**Advantages of M3NAS**. The above M3NAS process has two advantages to facilitate the search for more diverse architectures. First, AANS selects parents according to the distance of the network architectures (XOR-distance and $\tau$-distance) to promote the generation of unseen architectures. Second, the last frontier of the newly generated population is selected based on maximizing the ADD so that more diverse architectures are retained in the evolutionary process, thereby ensuring the retention of richer architectures in the final Pareto networks.

### E. NDH and SE

Since NAS is a nonbenchmark optimization problem, its true Pareto set is difficult to obtain. Hence, the difference between the searched networks and the global optimal networks cannot be calculated. As a result, considering the modular characteristics of MoTS-Net, the NDH and SE are proposed to reflect the architectural properties of the searched networks.

**NDH**. The NDH is presented in the form of a heatmap, where the frequency at which node $j$ ($j = 0, 1, \ldots, 4$) is selected in the $L_i$-th layer ($i = 2, 3, \ldots, p-1$) is shown on the NDH. Therefore, the darker/lighter the colour of a block in the NDH is, the higher/lower the frequency of the corresponding node in that layer is. Furthermore, the more uniform the colour distribution in the NDH is, the stronger the diversity of the network architectures is.

**SE**. Although the NDH can visually reflect the diversity of architectures, it cannot support quantitative comparison and analysis. To address this issue, we introduce a structural entropy metric to quantitatively measure architectural diversity within a population. Specifically, $p_{i,j}$ denotes the probability that node $j \in \{0, 1, ..., 4\}$ is selected at layer $L_i$, where $i \in \{2, 3, ..., w-1\}$. This probability is approximately estimated by the frequency of node selection across all individuals in the population. Next, the entropy at layer $L_i$ is defined as (2).

$$H_i = -\sum_{j=0}^{4} p_{i,j} \log_5(p_{i,j}) \tag{2}$$

where $H_i \in [0, 1]$, and $\log_5$ ensures normalization since each layer has 5 candidate nodes. If a node is never selected, its corresponding term is defined as zero. The overall SE is then the average entropy across all the searchable layers, which is shown in (3).

$$SE = \frac{1}{w-2} \sum_{i=2}^{w-1} H_i \tag{3}$$

This metric quantitatively measures the spread of architecture choices across the population. A higher SE indicates that more nodes are being utilized uniformly across layers, reflecting greater architectural diversity.

### F. Discussion of Computational Efficiency

The computational efficiency of M3NAS is analysed in detail as follows. First, the computational complexities of the XOR-distance and $\tau$-distance are $O(w)$ and $O(wq^2)$, where $w$ and $q$ are the total number of layers in MoTS-Net and the number of candidate nodes of each layer, respectively. Afterwards, in each generation of M3NAS, the XOR-distance and $\tau$-distance are employed in the AANS strategy, which requires $O(N(w + wq^2 - 2q^2) + N^2)$ operations, and $N$ is the population size. The crossover and mutation need $O(w)$ operations, and the complexity of fast nondominated sorting is $O(mN^2)$, where $m$ is the number of objectives. The ADD also needs to calculate the XOR-distance and $\tau$-distance, whose complexity is $O(N(w+wq^2-2q^2))$. Finally, the elite selection needs $O(N^2)$ operations to choose the next generation of individuals. As a result, the overall computational complexity of each generation in M3NAS is $O(wq^2N + mN^2)$.

## IV. Experimental Design

### A. Peer Competitors

To validate the effectiveness of our approach, we compare M3NAS-TSF against six representative methods. The CNN serves as a baseline for local pattern extraction, while the LSTM represents a classic recurrent model for capturing long-term dependencies. LSTNet [5] combines convolutional and recurrent networks to model both short-term and long-term patterns. Informer [6] employs a ProbSparse self-attention mechanism for efficient long-sequence forecasting. Crossformer [43] enhances transformers by modelling cross-time and cross-dimension dependencies. SOFTs [44] utilizes a lightweight multilayer perceptron-based architecture with linear complexity for channel interactions.

### B. Benchmark Datasets

In this work, we use five benchmark datasets with four forecasting horizons in the univariate and multivariate forecasting modes (a total of 40 cases) to evaluate our proposed algorithm. 1) Beijing PM2.5 [45] contains the PM2.5 data with an hour sampling rate from the US Embassy in Beijing from January 1, 2010 to December 31, 2014, where the PM2.5 concentration is our forecasting target. 2) Bike Sharing [46] contains the hourly and daily counts of rental bikes between the years 2011 and 2012 in the bike-share system with the corresponding weather and seasonal information, where the total number of rental bicycles per hour is our forecasting target. 3) The electricity transformer temperature (ETT) [6] is a crucial indicator of long-term electric power deployment, which consists of 2 years of data from two separate counties in China and is divided into two subsets {ETTh1, ETTh2} at the 1-hour level. The oil temperatures of ETTh1 and ETTh2 are our forecasting targets. 4) S&P500 [47] contains historical stock index data from January 29, 1993 to September 29, 2016, including a volatility index that reflects market uncertainty and is often regarded as the market's "fear index". The closing value of this volatility index serves as our forecasting target.

### C. Evaluation Metrics

**Evaluation Metrics of Forecasting Performance**. In this work, two commonly used metrics, the root mean square error (RMSE) and mean absolute error (MAE), are employed to quantitatively assess the forecasting accuracy of the TSF models.

**Evaluation Metrics of Model Complexity and Inference Efficiency**. To assess the computational efficiency of the searched TSF models, the number of parameters and Flops are reported to assess the model size. Furthermore, four runtime-level metrics are employed to provide a practical evaluation of inference efficiency in real-world scenarios.

1) Inference Time (IT): This metric measures the total latency of a single forwards pass for all the input samples. It captures how quickly the model can generate predictions, which is especially crucial in latency-sensitive applications such as edge computing.
2) Inference Time per Sample (ITS): This metric measures the average latency of a single forwards pass for one input sample, whose value is equal to the value of the IT divided by the total number of samples.
3) Peak GPU Memory Usage (PGMU): This metric refers to the maximum amount of GPU memory allocated during inference, monitored using `nvidia-smi`. It reflects the memory efficiency of the model and is critical for deployment in environments with constrained GPU resources.
4) CPU Utilization (CU): This metric represents the average CPU usage during inference and is calculated over a 5-second window using the `psutil` library. It helps evaluate the extent to which the model places a computational burden on the host processor, which is particularly meaningful when GPU acceleration is limited or unavailable.

To ensure fair and stable measurements, all the values are averaged over 1,000 independent inference runs, following 20 warm-up passes. This setup effectively mitigates initialization overhead and transient fluctuations, yielding more reliable and reproducible efficiency metrics.

### D. Experimental Details

M3NAS-TSF is completely based on the known input in a lookback window and under four direct forecast horizons $L_y = \{48, 96, 168, 192\}$. The look-back window size $L_x$ is fixed to 50. The converted image size $S_{im}$ is set to 32. The crowding factor $F_c$ of M3NAS is 0.4. The maximum evolutionary generation $g$ is determined to be 30 through a parameter analysis experiment. The population size $N$, crossover probability $p_c$, and mutation probability $p_m$ are set to 30, 0.8, and 0.2, respectively. The total number of network layers in MoTS-Net $w$ is set to 10. Moreover, all experiments except for those related to inference efficiency were conducted on a single NVIDIA GeForce RTX 3090 24 GB GPU.

## V. Experimental Results

### A. Comparative Analysis of Forecasting Performance

Tables III and IV summarize the forecasting performance results in the univariate forecasting mode and the multivariate forecasting mode, respectively. For M3NAS-TSF, we report the mean and standard deviation of the best-performing network (M3NAS-TSF$_{RMSE}$) from three independent runs. Statistical significance ($p < 0.05$) was verified through one-sample t-tests. The best and second-best RMSE and MAE values in each row are highlighted in **bold** and underlined, respectively.

From Tables III, i.e., in the univariate forecasting mode, M3NAS-TSF achieves the best performance on most datasets and horizons, particularly under long-term settings ($L_y = 168, 192$). Specifically, on PM2.5, it reduces the RMSE by 0.36% and 0.04% at $L_y = 168$ and 192, respectively. On Bike, the reduction in the RMSE reaches 1.39%, 2.67%, and 1.91% over the second-best method at $L_y = 96, 168$, and 192. With respect to ETTh1, M3NAS-TSF achieves a consistent reduction in the RMSE ranging from 1.70% to 2.94%. On S&P500, the RMSE decreases by a maximum of 12.93%. From Tables IV, i.e., in the multivariate forecasting mode, M3NAS-TSF continues to outperform at longer horizons. For PM2.5 and ETTh1, the RMSE is reduced by 0.28%-2.90%. On S&P500, the maximum reduction in the RMSE is 14.50%.

The above results demonstrate the forecasting effectiveness of M3NAS-TSF, particularly in long-term TSF tasks. This improvement can be attributed to its use of time series-to-image encoding. The combination of four complementary encoding methods enables the model to effectively capture and retain both global trends and periodic structures, thereby enhancing its ability to model long-range dependencies. For the current representative TSF models, although Informer comprises quite a complex deep network, its key modules, such as self-attention distilling, are designed for handling extremely long outputs ($L_y = 336, 720, 960$). Thus, its performance is not prominent under the settings of this work. Moreover,

TABLE III
PERFORMANCE SUMMARY OF UNIVARIATE FORECASTING.

| Methods | | LSTM | | CNN | | LSTNet | | Informer | | Crossformer | | SOFTS | | M3NAS-TSF$_{RMSE}$ | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Metrics | | RMSE | MAE | RMSE | MAE | RMSE | MAE | RMSE | MAE | RMSE | MAE | RMSE | MAE | RMSE | MAE |
| Datasets | $L_y$ | | | | | | | | | | | | | | |
| PM2.5 | 48 | 82.67 | **56.54** | 96.72 | 65.03 | 92.99 | 68.51 | 84.16 | 61.21 | **79.61** | 57.59 | 86.76 | 60.81 | 89.41±0.27 | 60.81±0.18 |
| | 96 | 90.02 | **60.92** | 93.38 | 68.08 | 93.99 | 68.08 | 89.44 | 64.71 | **86.38** | 62.82 | 96.99 | 68.64 | 90.01±0.23 | 61.41±0.21 |
| | 168 | 93.40 | 62.54 | 94.36 | 71.38 | 222.81 | 201.87 | 90.71 | 67.00 | 90.24 | 64.95 | 100.57 | 70.79 | **89.91±0.59** | **61.78±0.33** |
| | 192 | 93.96 | 62.76 | 93.42 | 68.61 | 369.12 | 349.81 | 91.42 | 65.87 | 90.93 | 65.51 | 101.19 | 71.18 | **90.89±0.46** | **61.84±0.24** |
| Bike | 48 | **129.29** | 106.55 | 263.86 | 190.94 | 172.61 | 117.39 | 144.72 | 95.35 | 158.20 | 107.93 | 136.24 | 101.01 | 137.68±0.32 | **94.34±0.16** |
| | 96 | 153.11 | 116.86 | 246.97 | 181.60 | 161.26 | 111.32 | 166.60 | 111.16 | 168.05 | 116.23 | 148.02 | 110.91 | **145.95±0.25** | **100.54±0.13** |
| | 168 | 162.42 | 114.42 | 244.00 | 180.94 | 458.27 | 395.30 | 184.44 | 125.53 | 168.25 | 116.10 | 156.99 | 112.30 | **152.79±0.51** | **105.59±0.21** |
| | 192 | 164.12 | 115.20 | 246.78 | 182.04 | 438.17 | 392.27 | 190.41 | 129.17 | 172.65 | 120.14 | 159.73 | 114.41 | **156.67±0.20** | **108.41±0.12** |
| ETTh1 | 48 | 3.26 | 2.75 | 7.63 | 6.85 | 3.32 | 2.61 | 2.62 | 2.14 | 3.06 | 2.54 | **1.86** | **1.46** | 3.55±0.00 | 2.92±0.00 |
| | 96 | 3.80 | 2.80 | 7.06 | 6.24 | 3.47 | 2.75 | 3.35 | 2.67 | 3.59 | 3.02 | 3.39 | 2.72 | **3.28±0.00** | **2.48±0.00** |
| | 168 | 3.74 | 2.75 | 7.09 | 6.24 | 13.93 | 13.30 | 3.43 | 2.79 | 4.14 | 3.51 | 3.44 | 2.90 | **3.37±0.00** | **2.71±0.00** |
| | 192 | 3.79 | **2.72** | 7.07 | 6.25 | 20.10 | 19.56 | 3.56 | 2.85 | 4.36 | 3.74 | 3.51 | 2.95 | **3.45±0.01** | 2.96±0.01 |
| ETTh2 | 48 | 6.05 | 4.73 | 24.34 | 22.05 | 7.18 | 5.59 | 5.10 | 3.92 | 5.71 | 4.39 | **3.55** | **2.74** | 8.02±0.43 | 6.10±0.15 |
| | 96 | 6.90 | 5.36 | 22.62 | 20.23 | 7.43 | 5.89 | 6.51 | 5.00 | 6.88 | 5.37 | **4.19** | **3.29** | 8.10±0.25 | 6.30±0.13 |
| | 168 | 8.43 | 7.82 | 22.71 | 20.26 | 43.77 | 42.46 | 7.01 | 5.46 | 7.49 | 5.93 | **4.71** | **3.74** | 8.12±0.02 | 6.56±0.01 |
| | 192 | 8.70 | 7.99 | 22.53 | 20.19 | 28.64 | 27.26 | 8.26 | 6.98 | 7.59 | 5.99 | **4.87** | **3.88** | 8.15±0.05 | 6.69±0.02 |
| S&P500 | 48 | 4.55 | 3.93 | 4.59 | 3.57 | 4.92 | 4.40 | 3.05 | 2.45 | **3.03** | 2.59 | **3.03** | 2.56 | 3.09±0.00 | **2.13±0.00** |
| | 96 | 5.22 | 3.74 | 4.02 | 3.18 | 4.68 | 4.20 | 3.91 | 2.92 | 3.70 | 3.11 | 3.22 | 2.70 | **3.10±0.01** | **2.15±0.00** |
| | 168 | 5.36 | 3.84 | 4.01 | 3.09 | 53.72 | 53.39 | 5.69 | 4.61 | 4.42 | 3.79 | 3.43 | 2.88 | **3.12±0.00** | **2.19±0.01** |
| | 192 | 5.39 | 3.88 | 3.92 | 2.98 | 28.95 | 28.63 | 5.43 | 3.97 | 4.68 | 4.04 | 3.51 | 2.95 | **3.04±0.00** | **2.12±0.00** |
| **$1^{st}$ Count** | | 0 | 3 | 0 | 1 | 0 | 0 | 0 | 0 | 3 | 0 | 7 | 5 | 11 | 12 |

TABLE IV
PERFORMANCE SUMMARY OF MULTIVARIATE FORECASTING.

| Methods | | LSTNet | | Informer | | Crossformer | | SOFTS | | M3NAS-TSF$_{RMSE}$ | |
|---|---|---|---|---|---|---|---|---|---|---|---|
| Metrics | | RMSE | MAE | RMSE | MAE | RMSE | MAE | RMSE | MAE | RMSE | MAE |
| Datasets | $L_y$ | | | | | | | | | | |
| PM2.5 | 48 | 79.58 | **55.74** | 80.87 | 60.38 | **79.10** | 56.60 | 84.58 | 60.23 | 89.39±0.32 | 59.94±0.19 |
| | 96 | 88.75 | 63.98 | 89.58 | 64.93 | **85.97** | **62.63** | 96.34 | 68.57 | 93.90±0.37 | 64.89±0.25 |
| | 168 | 96.96 | 64.70 | 98.77 | 70.80 | 98.81 | 67.34 | 116.95 | 91.95 | **96.30±0.45** | **64.65±0.29** |
| | 192 | 97.51 | 65.21 | 99.76 | 70.90 | 99.31 | 67.08 | 117.06 | 92.02 | **95.84±0.47** | **63.71±0.21** |
| Bike | 48 | **107.64** | **69.39** | 167.06 | 112.32 | 158.39 | 108.14 | 116.30 | 81.08 | 137.80±0.51 | 93.90±0.21 |
| | 96 | **126.56** | **84.73** | 180.77 | 122.32 | 167.66 | 116.61 | 128.29 | 91.24 | 145.06±0.66 | 99.39±0.27 |
| | 168 | 135.33 | **91.51** | 201.54 | 139.05 | 170.15 | 118.08 | **131.79** | 92.13 | 153.60±0.60 | 106.01±0.31 |
| | 192 | **130.94** | **87.82** | 198.54 | 137.10 | 170.67 | 119.28 | 134.59 | 94.41 | 155.64±0.59 | 110.47±0.30 |
| ETTh1 | 48 | 3.64 | 2.83 | 6.11 | 5.40 | 3.00 | 2.47 | **2.84** | **2.44** | 3.18±0.01 | 2.55±0.01 |
| | 96 | 4.60 | 3.69 | 6.89 | 5.97 | 3.49 | 2.88 | **3.17** | **2.70** | 3.36±0.00 | 2.72±0.00 |
| | 168 | 5.46 | 4.48 | 6.67 | 5.73 | 3.95 | 3.32 | 3.44 | 2.89 | **3.34±0.00** | **2.70±0.00** |
| | 192 | 4.96 | 3.92 | 6.64 | 5.69 | 4.07 | 3.44 | 3.54 | 2.95 | **3.53±0.01** | **2.82±0.01** |
| ETTh2 | 48 | 7.53 | 5.32 | 6.77 | 5.30 | 5.99 | 4.78 | **3.56** | **2.74** | 9.37±0.09 | 8.28±0.06 |
| | 96 | 9.74 | 7.20 | 8.69 | 6.85 | 6.96 | 5.58 | **4.22** | **3.29** | 10.02±0.01 | 8.50±0.00 |
| | 168 | 11.52 | 8.68 | 10.44 | 8.63 | 7.49 | 6.06 | **4.79** | **3.81** | 10.11±0.02 | 8.52±0.01 |
| | 192 | 11.98 | 8.75 | 10.76 | 8.75 | 7.54 | 6.09 | **4.92** | **3.93** | 10.63±0.01 | 8.66±0.01 |
| S&P500 | 48 | 10.77 | 9.98 | **3.28** | **2.34** | 9.53 | 9.02 | 4.11 | 3.62 | 3.86±0.00 | 3.16±0.00 |
| | 96 | 12.55 | 11.34 | 4.01 | 2.96 | 8.64 | 8.19 | 4.27 | 3.73 | **3.92±0.00** | **2.93±0.00** |
| | 168 | 26.24 | 24.66 | 4.52 | 3.96 | 8.20 | 7.67 | 4.48 | 3.93 | **3.83±0.01** | **2.91±0.00** |
| | 192 | 22.28 | 20.42 | 4.12 | 3.89 | 8.72 | 8.19 | 4.55 | 4.01 | **3.97±0.01** | **3.22±0.01** |
| **$1^{st}$ Count** | | 3 | 5 | 1 | 1 | 2 | 1 | 7 | 6 | 7 | 7 |

although LSTNet is more competitive than Informer in the multivariate forecasting mode, its convolutional layers for discovering the local dependency patterns among multivariate features limit its performance in the univariate forecasting mode. Although Crossformer is designed to capture cross-time and cross-dimension dependencies, it relies on coarse-grained hierarchical partitioning, which limits its effectiveness in moderate-length and univariate settings. Similarly, SOFTS performs well on highly regular data but lacks mechanisms for modelling local temporal variations, leading to degraded performance in some scenarios.

### *B. Model Complexity and Inference Efficiency Analysis*

**Comparative Analysis of Model Complexity**. To compare the sizes of the TSF models, taking Bike with $L_y = 96$ in the univariate forecasting mode as an example, we visualize different TSF networks in terms of Flops, RMSE, and the number of parameters in Fig. 7, where M3NAS-TSF$_{RMSE}$, M3NAS-TSF$_{Flops}$, and M3NAS-TSF$_{knee}$ are the network with the smallest RMSE, the network with the smallest Flops, and the network located in the knee point searched by M3NAS, respectively. Although the CNN and the LSTM have the lowest model Flops, their RMSE values are consistently higher than those of the three proposed variants (M3NAS-TSF$_{RMSE}$, M3NAS-TSF$_{Flops}$, and M3NAS-TSF$_{knee}$). More-

over, although the current TSF models, such as Crossformer and Informer, have ingenious network architectures, they also have high Flops and suboptimal forecasting performance. Overall, the networks searched by M3NAS-TSF achieve state-of-the-art forecasting performance with fewer Flops and can be carried out efficiently in different scenarios.

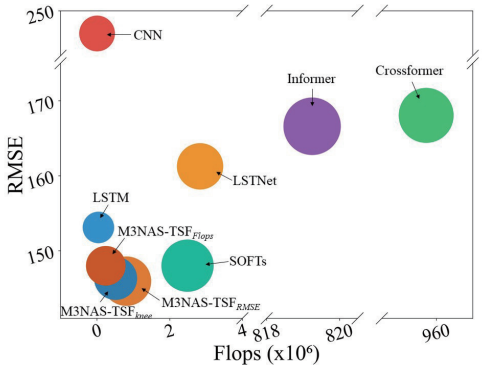



Fig. 7. Comparison results of nine TSF networks in terms of model Flops, RMSE, and the number of parameters on Bike with $L_y = 96$ in the univariate forecasting mode. The circle diameters are positively correlated with the number of model parameters.

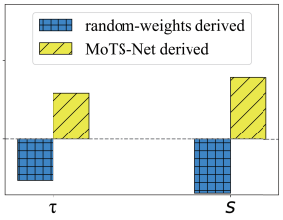
**Inference Efficiency on Resource-Constrained Devices**. To further evaluate the deployment ability of the proposed models in practical environments, we conduct a comprehensive comparison of inference efficiency on the following four common hardware platforms.

1) # Device 1: Intel Core i5-10210U CPU with 4 cores and 8 threads (low-power CPU).
2) # Device 2: Intel Core i9-14900KF CPU with 24 cores and 32 threads (high-performance CPU).
3) # Device 3: NVIDIA GeForce MX330 GPU with 2 GB VRAM (entry-level GPU).
4) # Device 4: NVIDIA GeForce RTX 4090 D GPU with 24 GB VRAM (high-performance GPU).

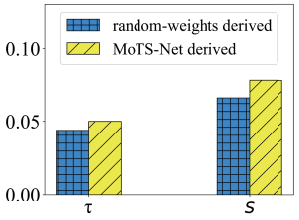
These devices cover a wide range of computational capabilities, from resource-constrained platforms to high-performance environments, allowing a comprehensive assessment of model inference efficiency. We measure four inference efficiency metrics and record the results in Table V. As shown in Table V, the models searched by M3NAS-TSF exhibit clear advantages on CPU platforms. On # Device 1, M3NAS-TSF$_{RMSE}$ achieves the fastest IT (5.00 ms) and ITS (1.50 ms) and the lowest CU (7.20%), while M3NAS-TSF$_{knee}$ ranks second. On # Device 2, M3NAS-TSF$_{knee}$ delivers the best performance in terms of the IT (2.57 ms), ITS (0.77 ms), and CPU load (8.56%). These results highlight the efficiency and deployment ability of our approach on resource-constrained or CPU-based devices.

### C. Estimated Performance Reliability Analysis of Subnetworks

Since the validity of the search stage in one-shot NAS relies on a high correlation between the super-net-derived performance and the fully trained performance (the performance refers to the RMSE value), we check their correlation with




(a) Bike with $L_y = 96$




(b) ETTh2 with $L_y = 96$

Fig. 8. The Kendall tau metric $\tau$ and the Spearman correlation coefficient $S$ between the random weight-derived performance/super-net-derived performance and the fully trained performance in the univariate forecasting mode.

the Spearman correlation coefficient $S$, and verify the ranking consistency with the Kendall tau metric $\tau$. The value of $S$ varies between -1 and +1, where positive values imply that two groups of variables are positively correlated, and vice versa. The value of $\tau$ also varies between -1 and +1, where $\tau$ close to 1 indicates a strong agreement for the correspondence between two rankings. Moreover, the random weight-derived performance is used for comparison.

As shown in Fig. 8, taking Bike with $L_y = 96$ in the univariate forecasting mode as an example, the $\tau$ and $S$ values of the MoTS-Net derived performance are much higher than those of the random weight-derived performance. This result means that the MoTS-Net can guarantee a certain degree of correlation between the super-net-derived performance and the fully trained performance. However, the $\tau$ and $S$ values on ETTh2 with $L_y = 96$ in the univariate forecasting mode are only slightly improved. The reason may be that MoTS-Net works for the single-path architecture search, while ignoring the combination of low-level features and high-level features, thus limiting the performance improvement on all the datasets to a certain extent.

### D. Diversity Analysis of Subnetworks

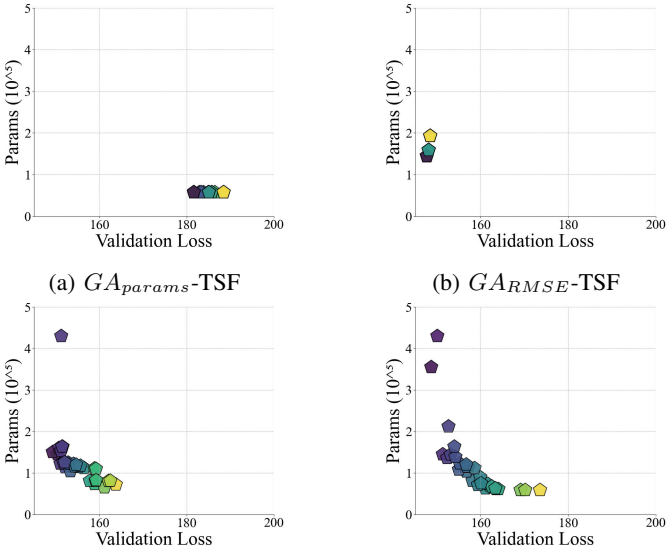
To further analyse the effectiveness of M3NAS, a careful ablation study is conducted on Bike with $L_y = 96$ in the univariate forecasting mode. We replace the search algorithm of M3NAS-TSF with NSGA-II and the genetic algorithm (GA). The detailed explanation is given as follows.

1) NSGA-II-TSF: The search algorithm in M3NAS-TSF is replaced with NSGA-II.
2) $GA_{RMSE}$-TSF: The search algorithm in M3NAS-TSF is replaced with the GA, where the objective of the GA is to minimize the forecasting error.
3) $GA_{params}$-TSF: The search algorithm in M3NAS-TSF is replaced with the GA, where the objective of the GA is to minimize the number of network parameters.

**Diversity Analysis in the Objective Space**. Figure 9 shows the final populations in the objective space searched by four methods ($GA_{params}$-TSF, $GA_{RMSE}$-TSF, NSGA-II-TSF, and M3NAS-TSF) under the same experimental setups. We find that the populations of (a) and (b) are clustered in a small region, because they are all obtained by the single-objective optimization algorithm, which considers only the

TABLE V
COMPARISON OF INFERENCE EFFICIENCY ON FOUR DIFFERENT DEVICES, WHERE THE MINIMUM AND SECOND MINIMUM VALUES IN EACH ROW ARE EMPHASIZED IN **BOLD**, UNDERLINED, RESPECTIVELY.

| Devices | Inference Efficiency | LSTNet | Informer | Crossformer | SOFTS | M3NAS-TSF$_{Flops}$ | M3NAS-TSF$_{RMSE}$ | M3NAS-TSF$_{knee}$ |
|---|---|---|---|---|---|---|---|---|
| # Device 1 | IT (s) | 17.23 | 143.17 | 191.64 | 11.10 | 6.70 | **5.00** | 5.19 |
| | ITS (ms) | 5.17 | 42.98 | 57.53 | 3.33 | 2.01 | **1.50** | 1.56 |
| | PGMU (MB) | - | - | - | - | - | - | - |
| | CU (%) | 24.58 | 68.17 | 57.30 | 21.02 | 9.77 | **7.20** | 7.65 |
| | TT (s) | 4315.64 | 35860.94 | 15963.36 | **384.49** | 593.43 | 825.08 | 645.73 |
| # Device 2 | IT (s) | 5.33 | 23.76 | 31.62 | 2.94 | 2.70 | 2.60 | **2.57** |
| | ITS (ms) | 1.60 | 7.13 | 9.49 | 0.88 | 0.81 | 0.78 | **0.77** |
| | PGMU (MB) | - | - | - | - | - | - | - |
| | CU (%) | 40.70 | 78.31 | 78.56 | 9.19 | 8.69 | 11.83 | **8.56** |
| | TT (s) | 9901.48 | 39606.30 | 2128.02 | **15.73** | 133.47 | 193.76 | 184.95 |
| # Device 3 | IT (s) | **5.87** | 126.28 | 369.88 | 7.37 | 21.62 | 20.16 | 20.56 |
| | ITS (ms) | **1.76** | 37.91 | 111.04 | 2.21 | 6.49 | 6.05 | 6.17 |
| | PGMU (MB) | **17.02** | 627.26 | 1299.68 | 20.27 | 517.56 | 519.51 | 517.64 |
| | CU (%) | **5.36** | 29.83 | 29.79 | 6.32 | 13.41 | 9.91 | 10.25 |
| | TT (s) | 1192.30 | 31026.66 | 3195.09 | **472.85** | 1017.44 | 3904.38 | 1338.07 |
| # Device 4 | IT (s) | **0.67** | 5.40 | 13.03 | 0.90 | 4.34 | 4.17 | 4.37 |
| | ITS (ms) | **0.20** | 1.62 | 3.91 | 0.27 | 1.30 | 1.25 | 1.31 |
| | PGMU (MB) | **16.99** | 629.88 | 1300.54 | 21.26 | 518.54 | 518.54 | 518.54 |
| | CU (%) | **3.22** | 5.69 | 6.71 | 4.38 | 5.81 | 5.44 | 5.73 |
| | TT (s) | 3864.11 | 28981.21 | 1064.02 | **4.81** | 61.86 | 51.79 | 56.88 |




(a) $GA_{params}$-TSF (b) $GA_{RMSE}$-TSF

(c) NSGA-II-TSF (d) M3NAS-TSF

Fig. 9. The searched final populations of four optimization algorithms in the objective space on Bike with $L_y = 96$ in the univariate forecasting mode.

number of parameters or the forecasting error and cannot provide decision-makers with networks balanced between different objectives. Moreover, the populations of (c) and (d) are distributed around a concave curve, whose distributions are more diverse, so decision-makers can choose suitable networks according to their requirements.

**Diversity Analysis in the Decision Space**. Figure 10 shows the NDHs and the SE values of the final populations searched by four methods ($GA_{params}$-TSF, $GA_{RMSE}$-TSF, NSGA-II-TSF, and M3NAS-TSF). We find that the NDHs of (a) and (b) are obviously darker on the right, i.e., the nodes of the networks prefer node 4. In particular, in $L_3, L_5, L_7$ and $L_9$ of (a), only node 4 is selected, because the stride of these layers is 1, among whose candidate nodes only node 4 has no trainable parameters. In addition, compared with (c), which has 10 nodes whose frequency is 0, (d) has 5 nodes whose frequency is 0, which shows that the networks in the population of (d) contain more types of candidate nodes, and (d) obtains a more diverse distribution in the decision space. Moreover, the SE values of (a)-(d) are 0.32, 0.38, 0.55, and 0.72, respectively. Their values gradually increase, indicating that node diversity is increasing, which is consistent with the visualization results of the NDH.

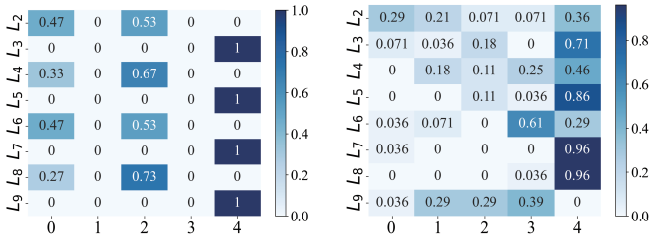
**Validation of Multimodal Solutions**. To verify that M3NAS yields multimodal solutions, i.e., distinct architectures with similar performance, we analyse two local regions of Pareto fronts on Bike with $L_y = 96$ and S&P500 with $L_y = 48$ in univariate forecasting, where these two local regions are delineated by the RMSE in $[167.50, 168.90]$ and the RMSE in $[6.30, 6.45]$, respectively. From Table VI, we observe that multiple architectures with different paths, such as $[4, 0, 2, 0, 1, 4, 2, 0]$ and $[3, 2, 1, 3, 4, 4, 4, 4]$, achieve near-consistent results for both RMSE and the number of parameters. This finding confirms that the proposed M3NAS successfully finds functionally equivalent yet architecturally diverse networks, thereby fulfilling the goal of multimodal searching.

### *E. Robustness Analysis in Fault Scenarios*

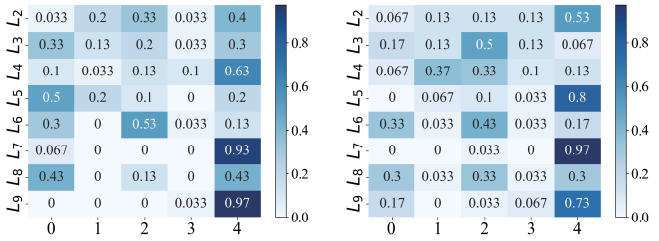
We conduct supplementary experiments on PM2.5 with $L_y = 168$ in the multivariate forecasting mode to evaluate the robustness of M3NAS-TSF under two typical failure scenarios: internal model parameter faults and input data missingness. The experimental results are presented in Tables VII and VIII, where the minimum and second minimum values in each column are emphasized in **bold**, underlined, respectively. We utilize the Pareto-optimal architectures searched by M3NAS-

TABLE VI
TWO GROUPS OF EXAMPLES OF DIFFERENT ARCHITECTURES WITH SIMILAR FORECASTING ERRORS AND NUMBERS OF PARAMETERS ON BIKE WITH $L_y = 96$ AND ON S&P500 WITH $L_y = 48$ IN UNIVARIATE FORECASTING.

| Datasets | $L_y$ | Path of networks | RMSE | Number of parameters ($10^5$) |
|---|---|---|---|---|
| **Bike** | 96 | [4, 0, 0, 0, 1, 4, 2, 2] | 168.12 | 1.43 |
| | | [4, 0, 2, 0, 1, 4, 2, 0] | 167.72 | 1.43 |
| | | [3, 0, 2, 0, 3, 4, 2, 2] | 167.65 | 1.45 |
| | | [3, 2, 1, 3, 4, 4, 4, 4] | 168.82 | 1.42 |
| **S&P500** | 48 | [1, 1, 3, 4, 0, 2, 4, 1] | 6.39 | 4.15 |
| | | [4, 3, 1, 4, 1, 4, 0, 1] | 6.44 | 4.13 |
| | | [4, 3, 3, 0, 3, 4, 0, 3] | 6.41 | 4.15 |



(a) $GA_{params}$-TSF, $SE = 0.32$ (b) $GA_{RMSE}$-TSF, $SE = 0.38$



(c) NSGA-II-TSF, $SE = 0.55$ (d) M3NAS-TSF, $SE = 0.72$

Fig. 10. The node distribution frequency heatmaps on Bike with $L_y = 96$ in the univariate forecasting mode, and the SE values of (a)-(d) are 0.32, 0.38, 0.55, and 0.72, respectively.

TSF and refer to their weighted ensemble as M3NAS-TSF$_{pool}$, where the weights are adaptively learned via a linear layer.

**Internal Model Parameter Faults**. We simulate soft errors by randomly zeroing out 1%, 5%, and 10% of the model weights in multiple TSF models. As shown in Table VII, under 1% faults, all the models perform similarly. At higher fault rates (5% and 10%), M3NAS-TSF$_{pool}$ gradually outperforms the others, achieving the best RMSE under 10% faults. This finding demonstrates that architectural diversity helps mitigate the impact of model parameter-level faults through complementary model behaviours.

**Input Data Missingness.** We simulate sensor failures by randomly discarding 1%, 5%, and 10% of input data with missing values imputed via linear interpolation. The results in Table VIII show that M3NAS-TSF$_{pool}$ does not outperform under complete or 1% missing data, as some Pareto architectures have weaker fitting ability compared to M3NAS-TSF$_{RMSE}$, whose prediction results will slightly dilute the overall performance. However, at missing rates of 10%, M3NAS-TSF$_{pool}$ achieves the best performance. This can be attributed to the diversity of the Pareto-optimal architectures, which enables the ensemble to capture complementary patterns and reduce over-reliance on any single architecture, thereby enhancing robustness to missing data.

### F. Hyperparameter Sensitivity Analysis

We analyse the sensitivity of M3NAS-TSF to two key hyperparameters: the crowding factor ($F_c$) and the total number of network layers in MoTS-Net ($w$).

**Crowding factor**. We vary the value of crowding factor $F_c$ from 0.1 to 0.9 on S&P500 with $L_y = 48$ in the univariate forecasting mode. As shown in Fig. 11, the forecasting accuracy of M3NAS-TSF$_{RMSE}$ improves until $F_c$ reaches 0.4, beyond which the performance decreases. This finding indicates that a moderate $F_c$ best balances architectural diversity and solution quality. Thus, $F_c = 0.4$ is adopted as the default.

**Total number of network layers**. We evaluate the MoTS-Net depth by varying $w$ from 6 to 14, i.e., the number of searchable layers varying from 4 to 12. The results in Fig. 12 show that the RMSE of the M3NAS-TSF$_{knee}$ is minimized at $w = 10$, i.e., the number of searchable layers is 8, while the number of Flops increases monotonically. Therefore, $w = 10$ is selected for an optimal accuracy-efficiency trade-off.

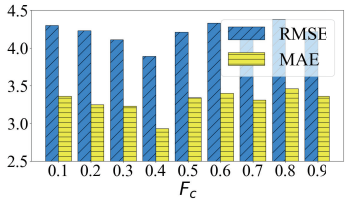



Fig. 11. Comparison in terms of the value of crowding factor on S&P500 with $L_y = 48$ in the univariate forecasting mode.

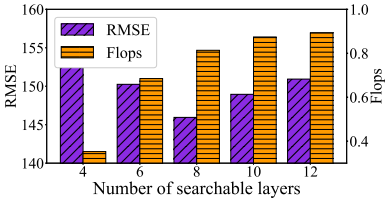



Fig. 12. Comparison in terms of the number of searchable layers on Bike with $L_y = 96$ in the univariate forecasting mode.

## VI. CONCLUSION

To address the practical demands of efficient and robust TSF models, this work proposes M3NAS-TSF, a multimodal multiobjective neural architecture search framework. The framework comprises a modular time series super-net MoTS-Net and a dedicated search algorithm M3NAS that minimizes both the forecasting error and the model size to discover Pareto-optimal architectures. In addition, the architecture-aware niching selection strategy and the architectural diversity distance metric in M3NAS explicitly promote architectural diversity in

TABLE VII
RMSE VALUES UNDER THE SCENARIO OF INTERNAL MODEL PARAMETER FAULTS ON PM2.5 WITH $L_y$ = 168 IN THE MULTIVARIATE FORECASTING MODE.

| Methods | | Complete data | Missing data | | |
|---|---|---|---|---|---|
| | | | Random 1% | Random 5% | Random 10% |
| Single model | LSTNet | 96.96 | 97.20 | **102.34** | 115.67 |
| | Informer | 98.77 | 98.41 | 105.12 | 118.93 |
| | Crossformer | 98.81 | 98.88 | 104.76 | 119.45 |
| | SOFTS | 116.95 | 116.02 | 120.57 | 128.31 |
| | M3NAS-TSF$_{RMSE}$ | **96.30** | **96.15** | 103.01 | 114.89 |
| Pool-based method | M3NAS-TSF$_{pool}$ | 101.43 | 101.92 | 102.58 | **113.74** |

TABLE VIII
RMSE VALUES UNDER MISSING DATA SCENARIOS ON PM2.5 WITH $L_y$ = 168 IN THE MULTIVARIATE FORECASTING MODE.

| Methods | | Complete data | Missing data | | |
|---|---|---|---|---|---|
| | | | Random 1% | Random 5% | Random 10% |
| Single model | LSTNet | 96.96 | 97.02 | **99.35** | 110.58 |
| | Informer | 98.77 | 98.06 | 101.98 | 110.37 |
| | Crossformer | 98.81 | 98.11 | 101.47 | 112.35 |
| | SOFTS | 116.95 | 119.34 | 118.36 | 123.09 |
| | M3NAS-TSF$_{RMSE}$ | **96.30** | **96.12** | 100.90 | 109.98 |
| Pool-based method | M3NAS-TSF$_{pool}$ | 101.43 | 101.55 | 100.01 | **107.65** |

the Pareto set to enhance failure resilience. Empirical results demonstrate that the diverse networks found by M3NAS-TSF achieve competitive forecasting accuracy with compact model sizes, offering decision-makers multiple robust alternatives to mitigate practical uncertainties.

Future research will explore several promising directions. First, designing a multi-path super-net tailored for time series data helps to mix multiscale temporal features to enhance the forecasting capability. Second, extending the current framework to dynamically adapt to concept drift scenarios such as consumer behaviour shifts is valuable for real-world robustness. Finally, investigating the integration of explainable AI techniques into the neural architecture search process could improve the trustworthiness of the searched architectures, which is crucial for high-stakes applications in the real world.

## REFERENCES


[1] M. Lu and X. Xu, "TRNN: An efficient time-series recurrent neural network for stock price prediction," *Inf. Sci.*, vol. 657, Feb. 2024, Art. no. 119951, doi:10.1016/j.ins.2023.119951.

[2] D. Li, "Mechanism-Empowered multivariate time series forecasting model: Application to tuberculosis prediction," *Knowledge-Based Syst.*, vol. 327, Oct. 2025, Art. no. 114124, doi: 10.1016/j.knosys.2025.114124.

[3] J. Zhang, S. Mao, L. Yang, W. Ma, S. Li, and Z. Gao, "Physics-informed deep learning for traffic state estimation based on the traffic flow model and computational graph method," *Inf. Fusion*, vol. 101, Jan. 2024, Art. no. 101971, doi: 10.1016/j.inffus.2023.101971.

[4] P. Gao, X. Yang, R. Zhang, P. Guo, J. Y. Goulermas, and K. Huang, "EgPDE-Net: Building continuous neural networks for time series prediction with exogenous variables," *IEEE Trans. Cybern.*, vol. 54, no. 9, pp. 5381–5393, Sep. 2024, doi:10.1109/TCYB.2024.3364186.

[5] G. Lai, W.-C. Chang, Y. Yang, and H. Liu, "Modeling long-and short-term temporal patterns with deep neural networks," in *Proc. Int. ACM SIGIR Conf. Res. Dev. Inf. Retr.*, Ann Arbor, USA, 2018, pp. 95–104.

[6] H. Zhou, S. Zhang, J. Peng, S. Zhang, J. Li, H. Xiong, and W. Zhang, "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, Online, 2021, pp. 11 106–11 115.

[7] R. C. Baumann, "Radiation-induced soft errors in advanced semiconductor technologies," *IEEE Trans. Device Mater. Reliab.*, vol. 5, no. 3, pp. 305–316, Sep. 2005, doi:10.1109/TDMR.2005.853449.

[8] G. Li, S. K. S. Hari, M. Sullivan, T. Tsai, K. Pattabiraman, J. Emer, and S. W. Keckler, "Understanding error propagation in deep learning neural network accelerators and applications," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Dallas, USA, 2018, pp. 1–12.

[9] K. Deb, A. Pratap, S. Agarwal, and T. Meyarivan, "A fast and elitist multiobjective genetic algorithm: NSGA-II," *IEEE Trans. Evol. Comput.*, vol. 6, no. 2, pp. 182–197, Apr. 2002, doi:10.1109/4235.996017.

[10] H. Pham, M. Guan, B. Zoph, Q. V. Le, and J. Dean, "Efficient neural architecture search via parameter sharing," in *Proc. Int. Conf. Mach. Learn.*, Stockholm, Sweden, 2018, pp. 4095–4104.

[11] H. Liu, K. Simonyan, and Y. Yang, "DARTS: Differentiable architecture search," in *Proc. Int. Conf. Learn. Represent.*, New Orleans, USA, 2019, pp. 1–13.

[12] S. You, T. Huang, M. Yang, F. Wang, C. Qian, and C. Zhang, "GreedyNAS: Towards fast one-shot NAS with greedy supernet," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Online, 2020, pp. 1999–2008.

[13] X. Chu, S. Lu, X. Li, and B. Zhang, "Mixpath: A unified approach for one-shot neural architecture search," in *Proc. IEEE/CVF Int. Conf. Comput. Vis.*, Paris, France, 2023, pp. 5972–5981.

[14] C. Yu, F. Wang, Z. Shao, T. Qian, Z. Zhang, W. Wei, Z. An, Q. Wang, and Y. Xu, "GinAR+: A robust end-to-end framework for multivariate time series forecasting with missing values," *IEEE Trans. Knowl. Data Eng.*, vol. 37, no. 8, pp. 4635–4648, May. 2025, doi:10.1109/TKDE.2025.3569649.

[15] Z. Ni, H. Yu, S. Liu, J. Li, and W. Lin, "Basisformer: Attention-based time series forecasting with learnable and interpretable basis," in *Proc. Adv. Neural Inf. Process. Syst.*, Vancouver, Canada, 2024, pp. 71 222–71 241.

[16] Q. Liu, H. Peng, L. Long, J. Wang, Q. Yang, M. J. Perez-Jimenez, and D. Orellana-Martin, "Nonlinear spiking neural systems with autapses for predicting chaotic time series," *IEEE Trans. Cybern.*, vol. 54, no. 3, pp. 1841–1853, Mar. 2024, doi:10.1109/TCYB.2023.3270873.

[17] R. Wan, S. Mei, J. Wang, M. Liu, and F. Yang, "Multivariate temporal convolutional network: A deep neural networks approach for multivariate time series forecasting," *Electronics*, vol. 8, no. 8, pp. 1–18, Aug. 2019, doi:10.3390/electronics8080876.

[18] F. Zhou, L. Li, K. Zhang, G. Trajcevski, F. Yao, Y. Huang, T. Zhong, J. Wang, and Q. Liu, "Forecasting the evolution of hydropower generation," in *Proc. Int. ACM SIGKDD Conf. Knowl. Discov. Data Min.*, Online, 2020, pp. 2861–2870.

[19] H. Han, Y. Liu, Y. Hou, and J. Qiao, "Data-driven robust multimodal multiobjective particle swarm optimization," *IEEE Trans. Syst., Man, Cybern.: Syst.*, vol. 54, no. 5, pp. 3231–3243, May. 2024, doi:10.1109/TSMC.2024.3357872.

[20] Z. Wang, Q. Ye, W. Wang, G. Li, and R. Dai, "A dynamic task-assisted constrained multimodal multi-objective optimization algorithm based on reinforcement learning," *Swarm Evol. Comput.*, vol. 98, Oct. 2025, Art. no. 102087, doi: 10.1016/j.swevo.2025.102087.

[21] Q. Dang, G. Zhang, L. Wang, S. Yang, and T. Zhan, "A generative adversarial networks model based evolutionary algorithm for multimodal multi-objective optimization," *IEEE Trans. Emerg. Top. Comput. Intell.*, vol. 1, no. 1, pp. 1–10, May. 2024, doi:10.1109/TETCI.2024.3397996.

[22] G. Li, W. Zhang, C. Yue, and G. G. Yen, "Clustering-based evolutionary algorithm for constrained multimodal multi-objective optimization," *Swarm Evol. Comput.*, vol. 91, Dec. 2024, Art. no. 101714, doi: 10.1016/j.swevo.2024.101714.

[23] C. Yue, B. Qu, and J. Liang, "A multiobjective particle swarm optimizer using ring topology for solving multimodal multiobjective problems," *IEEE Trans. Evol. Comput.*, vol. 22, no. 5, pp. 805–817, Oct. 2018, doi:10.1109/TEVC.2017.2754271.

[24] Y. Liu, H. Ishibuchi, Y. Nojima, N. Masuyama, and Y. Han, "Searching for local pareto optimal solutions: A case study on polygon-based problems," in *Proc. IEEE Congr. Evol. Comput.*, Wellington, New Zealand, 2019, pp. 896–903.

[25] J. Cao, Z. Qi, Z. Chen, and J. Zhang, "A multi-modal multi-objective evolutionary algorithm based on scaled niche distance,"

*Appl. Soft. Comput.*, vol. 152, Feb. 2024, Art. no. 111226, doi: 10.1016/j.asoc.2023.111226.

[26] Q. Gu, Y. Niu, Z. Hui, Q. Wang, and N. Xiong, "A hierarchical clustering algorithm for addressing multi-modal multi-objective optimization problems," *Expert Syst. Appl.*, vol. 264, Mar. 2025, Art. no. 125710, doi: 10.1016/j.eswa.2024.125710.

[27] Q. Dang, Q. Liu, S. Yang, and X. He, "Data-driven evolutionary algorithm based on inductive graph neural networks for multimodal multi-objective optimization," *IEEE Trans. Evol. Comput.*, vol. 1, no. 1, pp. 1–13, Feb. 2025, doi:10.1109/TEVC.2025.3541046.

[28] X. Gao, W. Bai, Q. Dang, S. Yang, and G. Zhang, "Learnable self-supervised support vector machine based individual selection strategy for multimodal multi-objective optimization," *Inf. Sci.*, vol. 690, Feb. 2025, Art. no. 121553, doi: 10.1016/j.ins.2024.121553.

[29] Z.-H. Zhan, J.-Y. Li, and J. Zhang, "Evolutionary deep learning: A survey," *Neurocomputing*, vol. 483, Apr. 2022, Art. no. 099, doi:10.1016/j.neucom.2022.01.099.

[30] Y. Oh, H. Lee, and B. Ham, "Efficient few-shot neural architecture search by counting the number of nonlinear functions," in *Proc. AAAI Conf. Artif. Intell.*, Philadelphia, USA, 2025, pp. 19 740–19 748.

[31] L. Ma, Y. Zhou, Y. Ma, G. Yu, Q. Li, Q. He, and Y. Pei, "Defying multi-model forgetting in one-shot neural architecture search using orthogonal gradient learning," *IEEE Trans. Comput.*, vol. 74, no. 5, pp. 1678–1689, May. 2025, doi:10.1109/TC.2025.3540650.

[32] Z. Guo, X. Zhang, H. Mu, W. Heng, Z. Liu, Y. Wei, and J. Sun, "Single path one-shot neural architecture search with uniform sampling," in *Proc. Eur. Conf. Comput. Vis.*, Online, 2020, pp. 544–560.

[33] Y. Liu, H. Ishibuchi, G. G. Yen, Y. Nojima, and N. Masuyama, "Handling imbalance between convergence and diversity in the decision space in evolutionary multimodal multiobjective optimization," *IEEE Trans. Evol. Comput.*, vol. 24, no. 3, pp. 551–565, Jun. 2020, doi:10.1109/TEVC.2019.2938557.

[34] Y. Li, J. Liu, and Y. Teng, "A decomposition-based memetic neural architecture search algorithm for univariate time series forecasting," *Appl. Soft Comput.*, vol. 130, Nov. 2022, Art. no. 109714, doi:10.1016/j.asoc.2022.109714.

[35] J.-Y. Li, Z.-H. Zhan, J. Xu, S. Kwong, and J. Zhang, "Surrogate-assisted hybrid-model estimation of distribution algorithm for mixed-variable hyperparameters optimization in convolutional neural networks," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 34, no. 5, pp. 2338–2352, May. 2021, doi:10.1109/TNNLS.2021.3106399.

[36] Y.-Q. Wang, J.-Y. Li, C.-H. Chen, J. Zhang, and Z.-H. Zhan, "Scale adaptive fitness evaluation-based particle swarm optimisation for hyperparameter and architecture optimisation in neural networks and deep learning," *CAAI Trans. Intell. Technol.*, vol. 8, no. 3, pp. 849–862, Jun. 2023, doi:10.1049/cit2.12106.

[37] I. Dumeur, S. Valero, and J. Inglada, "Paving the way toward foundation models for irregular and unaligned satellite image time series," *IEEE Trans. Geosci. Remote Sensing*, vol. 63, Jul. 2025, Art. no. 4414621, doi:10.1109/TGRS.2025.3589013.

[38] H. Zhu, Y. Zhu, J. Xiao, T. Xiao, Y. Ma, Y. Zhang, and F. Dai, "Exact: Exploring space-time perceptive clues for weakly supervised satellite image time series semantic segmentation," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Nashville, USA, 2025, pp. 14 036–14 045.

[39] Z. Wang and T. Oates, "Imaging time-series to improve classification and imputation," in *Proc. Int. Conf. Artif. Intell.*, Austin, USA, 2015, pp. 3939–3945.

[40] J. P. Eckmann, S. O. Kamphorst, and Ruelle, "Recurrence plots of dynamical systems," *World Sci. Ser. Nonlinear Sci., Ser. A*, vol. 16, pp. 441–446, 1995, doi:10.1142/9789812833709_0030.

[41] K. He, X. Zhang, S. Ren, and J. Sun, "Identity mappings in deep residual networks," in *Proc. Eur. Conf. Comput. Vis.*, Amsterdam, The Netherlands, 2016, pp. 630–645.

[42] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, "Densely connected convolutional networks," in *Proc. IEEE/CVF Conf. Comput. Vis. Pattern Recognit.*, Honolulu, USA, 2017, pp. 4700–4708.

[43] Y. Zhang and J. Yan, "Crossformer: Transformer utilizing cross-dimension dependency for multivariate time series forecasting," in *Proc. Int. Conf. Learn. Represent.*, Kigali, Rwanda, 2023, pp. 1–21.

[44] L. Han, X.-Y. Chen, H.-J. Ye, and D.-C. Zhan, "SOFTS: Efficient multivariate time series forecasting with series-core fusion," in *Proc. Adv. Neural Inf. Process. Syst.*, Vancouver, Canada, 2024, pp. 64 145–64 175.

[45] X. Liang, T. Zou, B. Guo, S. Li, H. Zhang, S. Zhang, H. Huang, and S. X. Chen, "Assessing Beijing's PM2.5 pollution: Severity, weather impact, apec and winter heating," *Proc. R. Soc. A.: Math. Phys. Eng. Sci.*, vol. 471, Oct. 2015, Art. no. 0257, doi:10.1098/rspa.2015.0257.

[46] H. Fanaee T and J. Gama, "Event labeling combining ensemble detectors and background knowledge," *Prog. Artif. Intell.*, vol. 2, no. 2, pp. 113–127, Jun. 2014, doi:10.1007/s13748-013-0040-3.

[47] G. H. Prices, "S&p 500 stock, Yahoo!" https://finance.yahoo.com/quote/%5EVIX/history?p=%5EVIX&guccounter=1, 2017.

**Yifan Li** received the B.S. degree in mathematics and applied mathematics from Xidian University, Xi'an, China, in 2018, and the Ph.D. degree in computer science and technology from Xidian University, in 2024. She is currently a lecturer with the Capital University of Economics and Business, Beijing, China.

Her current research interests include evolutionary computation, neural architecture search, and time seris analysis.

**Hong Zhao** (Member, IEEE) received the Ph.D. degree in South China University of Technology, Guangzhou, in 2020. She is currently an associate professor with the school of Guangzhou Institute of Technology, Xidian University.

Her current research interests include artificial intelligence, evolutionary computation, and their applications in design and optimization.

**Jian-Yu Li** (Member, IEEE) received the bachelor's and Ph.D. degrees in computer science andtechnology from the South China University ofTechnology, Guangzhou, China, in 2018 and 2022,respectively.

His research interests mainly include computational intelligence, data-driven optimization,machine learning, including deep learning, andtheir applications in real-world problems, and inenvironments of distributed computing and bigdata.

**Jing Liu** (Senior Member, IEEE) received the B.S. degree in computer science and technology and the Ph.D. degree in circuits and systems from Xidian University in 2000 and 2004, respectively. In 2005, she joined Xidian University as a lecture, and was promoted to a full professor in 2009. From Apr. 2007 to Apr. 2008, she worked at The University of Queensland, Australia as a postdoctoral research fellow, and from Jul. 2009 to Jul. 2011, she worked at The University of New South Wales at the Australian Defence Force Academy as a research associate. Now, she is a full professor in the Guangzhou Institute of Technology, Xidian University.

Her research interests include evolutionary computation, complex networks, fuzzy cognitive maps, multiagent systems, and data mining.